

TALLER EVALUATIVO

Programación Orientada a Objetos en Java

Asignatura:	Programación Orientada a Objetos
Docente:	_____
Estudiante:	_____
Fecha:	_____

Instrucción de entrega: La solución debe presentarse de manera individual. Cada estudiante debe crear una rama asociada al repositorio en GitHub donde se encuentra alojado este documento, desarrollar allí sus respuestas y evidencias, y realizar la entrega según las indicaciones del docente.

Pregunta 1. Estado actual de la programación de computadores en 2026

A partir de los reportes actuales sobre el estado de la programación de computadores en 2026, tales como encuestas globales de desarrolladores, reportes de plataformas de desarrollo colaborativo, estudios sobre inteligencia artificial aplicada a la programación y tendencias en lenguajes de programación, construya una posición personal argumentada frente al papel del programador moderno. En su respuesta, analice el avance de la inteligencia artificial, los asistentes de código, los lenguajes de programación actuales, la calidad del software y la necesidad de conservar el pensamiento lógico, el análisis de problemas y el enfoque orientado a objetos en la formación universitaria.

Pregunta 2. Sistemas de control de versiones en proyectos de software

Defina cuál es la principal funcionalidad de un sistema de control de versiones y explique su importancia dentro del desarrollo de software. Mencione herramientas de control de versiones como Git, Apache Subversion o Mercurial, y diferéncielas de plataformas de alojamiento y colaboración de código como GitHub, GitLab o Bitbucket. Finalmente, describa cómo utilizaría estas herramientas en un proyecto de software desarrollado por un equipo, considerando la creación del repositorio, el uso de ramas, confirmaciones de cambios, integración de aportes, revisión de código y recuperación de versiones anteriores.

Pregunta 3. Teoría de Java: JVM, JDK y evolución del lenguaje

Explique cómo funciona el proceso de desarrollo y ejecución de un programa escrito en Java, teniendo en cuenta la relación entre el código fuente, el compilador, el bytecode, la JVM y el JDK. Diferencie claramente qué es la JVM, qué es el JDK y cuál es el papel de cada uno dentro del ecosistema Java. Además, explique cómo funciona el esquema de versionamiento del lenguaje Java, diferenciando entre versiones LTS y no LTS, y analice la evolución del lenguaje desde sus primeras versiones hasta el contexto actual.

Pregunta 4. Programación básica en Java: ciclos y condicionales

Analice el siguiente problema básico de programación y formule una posible estrategia de implementación en Java utilizando ciclos y condicionales: un docente desea registrar las calificaciones de un grupo de estudiantes en una asignatura. El programa debe solicitar la cantidad de estudiantes, pedir la nota de cada uno, validar si la nota se encuentra entre 0.0 y 5.0, calcular el promedio general del grupo e indicar cuántos estudiantes aprobaron y cuántos reprobaron. Se considera que un estudiante aprueba si su nota es mayor o igual a 3.0. Explique las entradas, variables, ciclo, condicionales, validaciones y salidas esperadas.

Pregunta 5. Diseño de diagramas de clases: identificación de clases, atributos y métodos

Analice el siguiente problema y proponga un diseño básico de clases para representarlo mediante un diagrama de clases UML: una universidad desea desarrollar un sistema para gestionar la información de sus cursos académicos. Cada curso tiene un nombre, un código, un número de créditos y un docente asignado. Cada docente tiene un nombre, un número de identificación, un correo institucional y puede

UNIVERSIDAD DE CALDAS
Programa de Ingeniería Informática

Taller Evaluativo - Programación Orientada a Objetos en Java

orientar uno o varios cursos. Los estudiantes tienen nombre, código estudiantil, correo electrónico y pueden matricularse en varios cursos. El sistema debe permitir registrar cursos, asignar docentes, matricular estudiantes y consultar la información básica de cada curso. Identifique clases, atributos, métodos y relaciones.

Pregunta 6. Encapsulamiento, getters, setters y modificadores de acceso en Java

Explique el concepto de encapsulamiento en Programación Orientada a Objetos y analice su importancia en el diseño de clases en Java. Considere el uso de atributos privados, métodos públicos, getters, setters y modificadores de acceso como `private`, `public`, `protected` y el acceso por defecto. A partir de una clase `CuentaBancaria`, que almacena titular, número de cuenta y saldo disponible, proponga una implementación donde el saldo no pueda modificarse directamente desde fuera de la clase y solo sea posible consultarlo, depositar o retirar dinero mediante métodos con validaciones.

Pregunta 7. Herencia en Java: uso de `extends` y `super` en roles de empleados

Explique el concepto de herencia en Programación Orientada a Objetos y describa cómo se implementa en Java mediante la palabra reservada `extends`. Analice también el uso de `super` para invocar constructores, atributos o métodos de una clase padre. A partir de un sistema de empleados, donde todos tienen nombre, identificación y salario base, proponga una solución con una clase general `Empleado` y clases especializadas como `Desarrollador`, `Vendedor` y `Administrativo`, indicando qué atributos y métodos serían comunes y cuáles serían específicos de cada rol.

Pregunta 8. Implementación de polimorfismo en Java aplicado a un ejemplo de animales

Explique el concepto de polimorfismo en Programación Orientada a Objetos y analice cómo puede aplicarse en Java mediante la sobrescritura de métodos. A partir de un sistema para representar animales, donde todos tienen nombre, especie y edad, pero cada tipo de animal emite un sonido diferente, proponga una estrategia donde exista una clase general `Animal` y clases específicas como `Perro`, `Gato`, `Vaca` y `Ave`. Explique cómo cada clase sobrescribe el método `hacerSonido()` y cómo se podría recorrer una colección de animales ejecutando el mismo método de forma polimórfica.

Pregunta 9. Uso de interfaces en Java aplicado al ejemplo de animales

Retome el caso de animales de la pregunta anterior y proponga una solución usando interfaces en Java para representar comportamientos comunes. El sistema debe permitir que algunos animales puedan emitir sonidos, otros puedan desplazarse y algunos puedan volar. Diseñe interfaces como `Sonoro`, `Desplazable` y `Volador`, indicando qué métodos tendría cada una y qué clases las implementarían mediante `implements`. Guíe paso a paso la estrategia de solución y explique por qué este diseño favorece la flexibilidad, la reutilización y el polimorfismo frente a una solución basada únicamente en herencia.

Pregunta 10. Interfaces gráficas en Java Swing: formulario de registro de empleado

Diseñe una interfaz gráfica básica en Java utilizando Java Swing para procesar un formulario de registro de empleados. La interfaz debe permitir ingresar nombre, identificación, cargo, salario y correo electrónico. Además, debe incluir botones para `Registrar`, `Limpiar` y `Salir`. Dibuje un bosquejo de la interfaz, identifique los componentes utilizados y explique la función de `JFrame`, `JPanel`, `JLabel`, `TextField`, `ComboBox` y `Button`. Finalmente, escriba o describa un ejemplo básico de código donde se evidencie la captura y procesamiento de los datos ingresados por el usuario.